TrustLens — AI That Fact-Checks Honestly

Capstone: AI Agents — Intensive Vibe Coding (Google × Kaggle) **Track:** Agents for Good

---

The Problem

We are drowning in claims. Every day, billions of people read assertions in news articles, social-media posts, and private messages — and the single hardest question is no longer *finding* information, but *trusting* it. Which parts of what I just read are **verified facts**, which are **opinions**, and which are simply **made up**?

Generative AI has made this worse, not better. Ask a typical chatbot "is this true?" and it answers with total confidence — sometimes inventing sources, fabricating quotes, or producing a plausible-sounding number with no evidence behind it. The very tools people reach for to check facts are themselves a leading source of confident misinformation.

This is a problem worth solving for the public good. Misinformation erodes public health (anti-vaccine rumors), distorts elections, and overwhelms the journalists whose job is to separate signal from noise. The people most exposed — non-expert readers and under-resourced newsrooms — are exactly the ones with the fewest tools.

TrustLens is built to do the opposite of a confident-but-careless chatbot. It never invents a source. It always shows its evidence. And it scores trust using a transparent, deterministic formula — never a number the AI made up.

The Solution

TrustLens is a single agentic system that automatically adapts to two audiences from one free-text input:

- **Public Mode** — the user pastes an article, a URL, or a claim. TrustLens breaks it into sourced facts, opinions, and unverified claims, then returns a clear verdict (`true` / `false` / `unverified` / `mixed`), a **trust score from 0 to 100**, and a plain-language, verdict-first explanation written in the user's own language.
- **Newsroom Mode** — the user types a broad topic or a monitoring request. TrustLens cross-references multiple sources and produces a structured **journalistic brief**: a summary, a breakdown of reliable versus questionable sources, the contradictions between them, and suggested editorial angles.

One agentic backend serves both. Differentiation happens through a **Router Agent** that analyzes the input and selects the mode — so the user never has to choose a setting; they just type.

Why a Multi-Agent System

Fact-checking is not a single LLM call — it is a pipeline of distinct jobs, each with different needs:

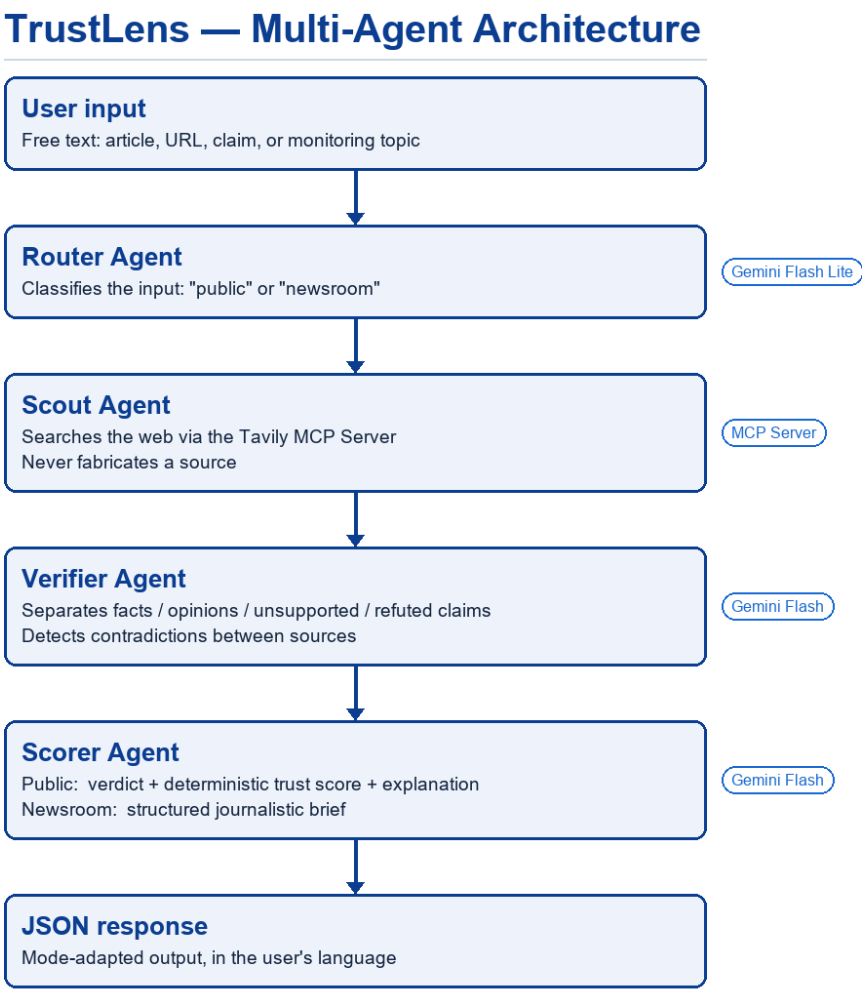
1. Understand what the user is actually asking for.

2. Search the live web for relevant, real sources.
3. Separate verified facts from opinions and unsupported claims.
4. Detect where sources contradict the claim or each other.
5. Score the result transparently.

Collapsing these into one prompt is precisely how hallucination creeps in: the model blurs "what I found" with "what I assumed." TrustLens instead models each job as a dedicated agent, orchestrated with **Google's Agent Development Kit (ADK)**. This separation gives three concrete benefits:

- **Grounding** — the search step is performed by a real external tool, not the model's memory.
- **Honesty** — the score is computed by code, never generated by the LLM.
- **Testability** — each small agent can be unit-tested independently against fixtures.

Architecture



The Four Agents

1. Router Agent. A lightweight classifier (Gemini Flash Lite) that reads the raw input and emits structured JSON: the mode (`public` or `newsroom`), a normalized search query, a confidence score, and the detected language (ISO 639-1). Choosing the fast, low-cost model here keeps latency and quota usage minimal for what is essentially a routing decision.

2. Scout Agent. The agent that grounds everything in reality. It collects sources through a **real Model Context Protocol (MCP) server** — the Tavily search server, spawned over stdio. In Public Mode it runs targeted searches around the claim; in Newsroom Mode it searches multiple angles of the topic. Critically, **if it finds nothing, it returns an empty list and says so** — it never fabricates a result. This is enforced at the code level: the MCP client catches every error path and returns `[]` rather than risking a hallucinated source.

3. Verifier Agent. The reasoning core (Gemini Flash, with a model fallback chain for resilience). It takes the original content and the collected sources and produces a structured analysis: factual claims with their supporting URLs and a confidence level; opinions; unsupported claims; **refuted claims** where sources directly contradict the content, along with what the sources actually say; and contradictions between sources. Its single non-negotiable rule, baked into the prompt: never invent a source or a claim.

4. Scorer / Final Writer Agent. Produces the final, mode-adapted output. In Public Mode it computes a verdict and a trust score, then asks the LLM to write *only* the human-readable explanation. In Newsroom Mode it writes the structured brief. The key safety design: **the LLM never produces the score itself.**

The Deterministic Trust Score

The trust score is the heart of the "honest by design" philosophy. It is computed entirely in code from the Verifier's structured output, using a documented formula:

- **Base** = (verified facts / total claims) × 100
- **Adjustments**: +2 per high-confidence fact (max +10), –2 per low-confidence fact (max –10), –5 per contradiction (max –15), –3 per unsupported claim (max –15)
- **Hard caps**: if any unsupported claims exist, the score is capped at 50; if there are unsupported claims *and* contradictions, capped at 25; if any refuted claims exist, capped at 15.

Because the number is deterministic, it is reproducible, auditable, and explainable — three properties that an LLM-generated score can never guarantee. The LLM's job is restricted to what it does well: writing clear prose. The arithmetic of trust stays in code.

Safety: Honest by Design

The capstone track is "Agents for Good," and for a fact-checking tool, "good" means *trustworthy*. Four guarantees are enforced in the implementation, not just promised in the prompt:

1. **Always cite sources.** Every output includes the URL and title of each source used. The user can click through and verify for themselves.
2. **Never hallucinate a source.** When the Scout Agent finds nothing, the system says so explicitly. The MCP client returns an empty list on any failure rather than inventing evidence.

3. **No hardcoded secrets.** All API keys are read from environment variables. Only `.env.example` is committed; real keys never enter the repository.
4. **Clear epistemic labels.** Every output distinguishes a *verified fact* from an *AI inference / opinion* from something *unverifiable* — so the user always knows the epistemic status of what they are reading.

Together these turn the usual generative-AI failure mode on its head: instead of confidently asserting, TrustLens transparently evidences.

Technology Stack

Component	Choice
Agent orchestration	Google ADK (Router, Scout, Verifier, Scorer)
LLM	Gemini — Flash Lite for routing/search, Flash for verification/scoring
Web search	Tavily MCP Server (real MCP client/server integration)
Backend API	FastAPI — a single `/analyze` endpoint plus `/health`
Frontend	Next.js 16 + React 19 + Tailwind CSS — one adaptive interface
Assisted development	Antigravity CLI (vibe-coding workflow)
Deployment target	Backend → Cloud Run; Frontend → Vercel

The backend exposes a single `POST /analyze` endpoint that accepts `{ "input": "string" }` and returns a mode-adapted JSON payload, with robust handling for empty input, oversized input (10,000-character cap), and a 120-second pipeline timeout. The orchestrator chains all four agents and includes retry logic with exponential backoff for transient model errors (rate limits, temporary unavailability) — important for staying within the free-tier quotas.

The frontend is deliberately minimal: a single input box. The user does not pick a mode or configure anything. They type, and the interface adapts its display based on the mode the backend returns — a fact-check card with a score in Public Mode, a formatted brief in Newsroom Mode.

How It Was Built

TrustLens was developed using a **vibe-coding** workflow with the **Antigravity CLI** as the assisted-development environment. Rather than hand-writing every line, the workflow centered on iterating — on agent prompts, on the deterministic scoring logic, and on edge-case handling — against real cases: a viral health myth and a live monitoring topic. Gemini served a dual role: as the

inference engine *inside* the agents, and as an iterative prompt-refinement tool *during* development, used to test classification variants and sharpen agent instructions.

This iterative, test-driven loop produced a backend with a full pytest suite — covering each agent, the end-to-end orchestrator, and the API contract — using JSON fixtures so the tests run deterministically without consuming API quota. That test discipline is what made it safe to refactor prompts aggressively while keeping the anti-hallucination guarantees intact.

Course Concepts Demonstrated

The capstone requires at least three course concepts; TrustLens demonstrates six:

1. **Multi-agent system** — four specialized ADK agents with a routing decision.
2. **MCP Server** — a genuine MCP client/server integration for web search (Tavily), not a raw API call.
3. **Antigravity** — the assisted development workflow used to build the project.
4. **Security features** — anti-hallucination guarantees, mandatory source citation, and deterministic (non-LLM) scoring.
5. **Deployability** — a documented path to Cloud Run (backend) and Vercel (frontend).
6. **Agent skills** — the Antigravity CLI workflow.

Impact and the "Agents for Good" Angle

TrustLens serves two audiences who are usually under-served. For the **general public**, it replaces the dangerous habit of asking a chatbot "is this true?" with a tool that refuses to bluff — giving a sourced verdict and a transparent score instead of a confident guess. For **newsrooms**, it offers a monitoring assistant that surfaces source reliability and contradictions, accelerating the slow, manual cross-referencing that fact-checkers do today.

The design choice that makes this "for good" is restraint: TrustLens is built around what AI should *not* do. It does not score with an LLM. It does not invent sources. It does not hide uncertainty. By engineering humility into the system — empty results when nothing is found, hard caps when claims are unsupported, explicit labels for every assertion — it models the kind of honest AI the information ecosystem actually needs.

Future Extensions

Several extensions are planned beyond this submission:

- **Image verification** — reverse image search to detect images reused in misleading contexts, pairing Gemini's multimodal capabilities with a reverse-image-search MCP server.
- **Deepfake / AI-generation detection** — a longer-term goal requiring specialized detection models.
- **Batch analysis** — letting newsrooms submit multiple articles or RSS feeds for continuous monitoring.

Conclusion

Misinformation is not solved by an AI that answers faster or more confidently — it is made worse by one. TrustLens demonstrates a different path: a multi-agent system where each step is grounded in a real tool, where trust is scored by transparent code rather than model intuition, and where the system's first instinct is to show its evidence and admit what it does not know. Verifiable, sourced, and honest by design — that is what an agent *for good* looks like.